Documento de continuidade — Conferência visual de recebimento

Projeto: Desafio Renault 2026 · **Repositório:** https://github.com/EnzoSenatori/DesafioRenault2026
De: Enzo Senatori · **Para:** Corrêa · **Data:** 12/10/2026
Ferramenta de continuidade: Codex

Este documento substitui o antigo `GUIA_PASSAGEM.md`. Ele junta o estado atual do projeto, as decisões e contratos do `CLAUDE.md` e as instruções para concluir os três próximos passos. Leia inteiro antes de começar. Os prompts para o Codex estão prontos na seção 7.

---

1. Resumo em uma página

O que o sistema faz: recebe uma foto dos itens recebidos e uma nota fiscal simulada em JSON. Um modelo YOLO hospedado no Roboflow detecta e conta os itens, o programa compara as contagens com a nota e gera:

- `relatorio.json`, com o status (`conforme`, `divergente` ou `erro`) e uma linha por item;
- `imagem_anotada.jpg`, a foto com as caixas e um painel "esperado × detectado × situação".

As ferramentas manuais fazem o papel das peças automotivas. O que se valida é a arquitetura.

Estado: o fluxo funciona de ponta a ponta e tem 39 testes automatizados, todos sem rede. O ponto fraco é a **precisão do modelo pré-treinado** nas nossas imagens (seção 4).

O que falta (os três próximos passos):

#	Passo	Depende de	Prioridade
1	Melhorar a detecção (avaliar outros modelos)	Chave do Roboflow e um gabarito das imagens	Alta
2	Modo câmera	Webcam	Média
3	Integração com o sistema do colega	Saber o que o outro sistema espera receber	Alta

---

2. Como rodar (Windows)

Requer Python 3.10 a 3.13. O `inference-sdk` 1.7.4 não suporta o 3.14, e o projeto foi testado com o 3.12.10.

```
git clone https://github.com/EnzoSenatori/DesafioRenault2026.git
cd DesafioRenault2026
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
copy .env.example .env
```

Preencha `ROBOFLOW_API_KEY` no `.env`. A chave fica no site do Roboflow, em *Settings → API Keys*. O `.env` nunca vai para o Git.

```
# Conferência de uma imagem contra uma nota
.venv\Scripts\python.exe src\conferir.py --imagem "imagens/teste/Img 16.jpg" --nota notas/nota_divergente.json

# Mesmo resultado, em JSON (para integração)
.venv\Scripts\python.exe src\conferir.py --imagem "imagens/teste/Img 16.jpg" --nota notas/nota_divergente.json --json

# Teste de fumaça: roda o modelo em todas as imagens de imagens/teste/
.venv\Scripts\python.exe src\testar_modelo.py

# Testes automatizados (não usam a internet)
.venv\Scripts\python.exe -m pytest
```

Código de saída do `conferir.py`: `0` conforme, `1` divergente, `2` erro.

3. Arquitetura e contratos

3.1 Módulos

Arquivo	Responsabilidade
`src/conferir.py`	CLI e funções de entrada `conferir()` e `conferir_quadro()`
`src/detector.py`	Chamada ao Roboflow e conversão para o formato interno; erros técnicos viram `ErroDeteccao`
`src/nota.py`	Leitura e validação da nota; erros viram `ErroEntrada`
`src/comparador.py`	Linhas, situações e status global
`src/visualizacao.py`	Caixas e painel com Pillow (o `cv2.putText` não desenha acentos)
`src/testar_modelo.py`	Teste de fumaça do modelo nas imagens de teste
`config.json`	Modelo, URL, limiar, classes ignoradas, de-para e nomes em português
`fixtures/`	Respostas reais do Roboflow, usadas nos testes
`tests/`	Testes `pytest` sem rede (clientes falsos simulam o Roboflow)

3.2 `config.json`

```
{
  "model_id": "tools-classification-byznk/2",
  "api_url": "https://serverless.roboflow.com",
  "limiar_confianca": 0.5,
  "classes_ignoradas": ["Fasteners"],
  "de_para": {"FER-001": "Pliers", "FER-002": "Screwdriver", "FER-003": "Wrench", "FER-004": "Hammer"},
  "nomes_exibicao": {"Pliers": "Alicate", "Screwdriver": "Chave de fenda", "Wrench": "Chave inglesa", "Hammer": "Martelo"}
}
```

Nenhuma parte do código depende das classes atuais. Para trocar de modelo, basta mudar `model_id`, `de_para` e `nomes_exibicao`.

3.3 Nota fiscal simulada

```
{
  "nota_id": "NF-DEMO-001",
  "fornecedor": "Fornecedor Exemplo Ltda.",
  "itens": [{"codigo": "FER-003", "descricao": "Chave inglesa", "quantidade": 1}]
}
```

- O código é tratado como texto.
- A quantidade deve ser um inteiro positivo; `2.0`, `"2"` e `true` são recusados.
- Linhas com o mesmo código são somadas.
- Um código fora do `de_para` e um JSON malformado geram erro explícito.

3.4 Formato interno de detecção

```
{"classe": "Wrench", "confianca": 0.57, "caixa_xyxy": [130.0, 87.0, 330.0, 500.0]}
```

A resposta real do Roboflow foi verificada (veja `fixtures/`). Cada predição traz `x` e `y` (o centro da caixa), `width` e `height` em pixels da imagem original, `confidence`, `class`, `class_id` e `detection_id`. O SDK aceita um array numpy na ordem **BGR** do OpenCV e reescala as caixas para o tamanho original.

3.5 Regras da comparação

Para cada classe esperada ou detectada (confiança de pelo menos 0,5 e fora de `classes_ignoradas`), `delta = detectado - esperado`.

Situação	Condição
`igual`	delta = 0
`falta`	delta < 0
`excesso`	delta > 0 e o item consta da nota
`nao_previsto`	detectado, mas ausente da nota

O status global segue estas regras:

- `conforme` só quando todas as linhas são `igual`.
- `erro` em qualquer falha técnica: rede, chave, imagem ou nota inválida. **Uma falha técnica nunca resulta em** `conforme`**.**
- `divergente` em todos os outros casos.

Se nenhuma detecção passar do limiar e a nota tiver itens, o relatório inclui o aviso "nenhuma ferramenta detectada — verifique a captura".

3.6 Relatório (contrato de integração — não quebre)

```
{
  "nota_id": "NF-DEMO-002",
  "imagem": "imagens/teste/Img 16.jpg",
  "status": "divergente",
  "linhas": [
    {"codigo": "FER-003", "item": "Chave inglesa", "classe": "Wrench",
     "esperado": 2, "detectado": 1, "delta": -1, "situacao": "falta"}
  ],
  "descartadas": [{"classe": "Hammer", "confianca": 0.31}],
  "avisos": [],
  "modelo_id": "tools-classification-byznk/2",
  "limiar_confianca": 0.5,
  "gerado_em": "2026-10-12T01:49:13-03:00"
}
```

- Em `erro`, `linhas` vem vazia e a causa, com o que fazer, está em `avisos[0]`.
- O `codigo` de uma linha `nao_previsto` vem do `de_para`, ou é `null` se a classe não tiver código.
- `conferir()` e `conferir_quadro()` **não lançam exceção por falha técnica**: devolvem `status: "erro"`.

3.7 Formas de chamar

```
import sys; sys.path.insert(0, "src")

# A partir de arquivos (grava relatorio.json e imagem_anotada.jpg)
from conferir import conferir
relatorio, pasta = conferir("foto.jpg", "nota.json")                # saidas/<data_hora>_<imagem>/
relatorio, pasta = conferir("foto.jpg", "nota.json", "minha/pasta")

# A partir de um quadro em memória (não grava nada)
from conferir import carregar_config, conferir_quadro
from nota import validar_nota
config = carregar_config()
nota = validar_nota(dados_da_nota, config["de_para"])               # lança nota.ErroEntrada se inválida
relatorio, evidencia_bgr = conferir_quadro(quadro_bgr, nota, config)
```

4. Diagnóstico do modelo atual

O modelo é o **"Tools Classification"**, de proelec3, publicado no Roboflow Universe sob licença CC BY 4.0, com arquitetura YOLOv8s: https://universe.roboflow.com/proelec3/tools-classification-byznk. A métrica publicada (mAP@50 de 98,7%) foi medida no conjunto do próprio autor.

No teste de fumaça com 21 imagens da internet (fotos de produto com fundo branco e baixa resolução), o resultado foi este:

Resultado	Imagens
Nenhuma detecção em nenhum nível de confiança	Img 1, 2, 9, 10, 11, 13, 14, 18, 19, 20
Só detecções abaixo de 0,5	Img 3, 4, 5, 6, 8, 12, 17, 21
Alguma detecção acima de 0,5	Img 7, 15, 16

Mesmo nas três imagens com detecção, a contagem não sai completa:

- **Img 7:** detectou 1 chave de fenda corretamente, mas marcou outras duas como um único "martelo". O martelo e os alicates da foto ficaram de fora.
- **Img 15:** contou 3 das 4 chaves inglesas.
- **Img 16:** detectou a chave inglesa. A chave de grifo ficou de fora, o que está correto, porque não é uma classe do modelo.
- **Img 3:** as 5 chaves de fenda foram classificadas como "Hammer", com confiança abaixo de 0,31.

Conclusão: o pipeline está correto, mas o modelo não generaliza para as nossas imagens. As notas de exemplo (`notas/nota_conforme.json` e `notas/nota_divergente.json`) foram feitas para a Img 16, a única com resultado coerente.

---

5. Decisões já tomadas

- Python 3.12.10 no `.venv`, com as versões fixadas no `requirements.txt` (`inference-sdk` 1.7.4, `opencv-python` 4.12.0.88, `python-dotenv` 1.2.4, `Pillow` 12.3.0, `pytest` 9.1.1).
- A chave vai no cabeçalho `Authorization` (`api_key_transport="header"`), não na URL. Ela nunca aparece no código, em logs ou nas saídas.
- O servidor devolve detecções a partir de 0,1 de confiança, para o relatório listar as descartadas. O corte real é o `limiar_confianca`.
- Confiança exatamente igual ao limiar conta.
- `Fasteners` é descartada por completo: não conta, não vira linha e não aparece em `descartadas` nem na imagem.
- Imagens com menos de 560 px de altura são ampliadas só na evidência. As coordenadas do relatório continuam sendo as da imagem original.
- `saidas/` fica no `.gitignore`. Os exemplos de evidência versionados estão em `docs/`.

---

6. Regras do projeto (valem também para o Codex)

Convenções de código

- Cada função Python leva um **comentário breve acima da definição** (`def`). **Não use docstrings.**
- Nomes de variáveis, funções e mensagens ficam em português.
- Todo texto desenhado em imagem passa pelo Pillow com uma fonte TTF, por causa dos acentos.

- Mensagens de erro dizem o que falhou e o que fazer.
- Não invente formato de resposta, versão de biblioteca ou resultado. Verifique.
- Todo teste novo roda sem rede. Use clientes falsos, como em `tests/test_conferir.py`.

Fora de escopo, salvo decisão da equipe: treino ou ajuste fino de modelo, Streamlit, banco de dados, leitura (OCR) da nota real, vídeo com contagem acumulada. Uma API web também estava fora de escopo, mas o passo 3 pode exigi-la (seção 7.4).

Armadilhas já encontradas no Windows

- Use sempre `.venv\Scripts\python.exe`. Em alguns terminais, `python` abre a Microsoft Store.
- Leia e grave imagens com `np.fromfile` + `cv2.imdecode` e `cv2.imencode` + `tofile`. Os nomes têm espaços (`Img 16.jpg`), e o `cv2.imread` falha com acentos.
- `load_dotenv()` sem caminho falha em scripts passados pela entrada padrão. Passe o caminho do `.env` de forma explícita, como faz o `detector.py`.
- O SDK emite um aviso sobre o envio da chave. É inofensivo; para silenciar, use `PYTHONWARNINGS=ignore`.

---

7. Os três próximos passos

7.1 Antes de tudo: preparar o Codex

O Codex lê instruções do arquivo `AGENTS.md`, não do `CLAUDE.md`. Comece criando esse arquivo com o prompt abaixo.

> **Prompt 0 — preparar o repositório**
>
> Leia `CONTINUIDADE.md` e `CLAUDE.md` por inteiro. Crie um `AGENTS.md` na raiz com as regras da seção 6 do `CONTINUIDADE.md` (convenções, escopo e armadilhas do Windows), os comandos para rodar a aplicação e os testes, e a instrução de nunca imprimir nem gravar a `ROBOFLOW_API_KEY`. Não altere nenhum outro arquivo. Rode `.venv\Scripts\python.exe -m pytest` e me mostre o resultado.

7.2 Passo 1 — Melhorar a detecção

Objetivo: encontrar um modelo pronto do Roboflow Universe que acerte as contagens nas nossas imagens e trocá-lo no `config.json`.

Por que primeiro: sem um modelo melhor, a demonstração só funciona em uma imagem.

Como fazer:

1. **Gabarito (tarefa humana, cerca de 15 minutos):** crie `imagens/teste/gabarito.json` com a contagem real de cada imagem, olhando as fotos. Exemplo: `{"Img 15.jpg": {"Wrench": 4}, "Img 16.jpg": {"Wrench": 1}}`. Conte só as classes do modelo e anote as ferramentas que não pertencem a nenhuma classe. O Codex não deve inventar o gabarito.
2. **Candidatos:** procure modelos de detecção de ferramentas em https://universe.roboflow.com, com buscas como "tools", "hand tools", "wrench hammer pliers screwdriver". Prefira modelos treinados com fotos variadas, não só de laboratório. Anote o `model_id` (`projeto/versão`) e as classes de cada um. Modelos públicos rodam com a nossa chave pelo mesmo endpoint.
3. **Avaliação:** o Codex cria um script que roda cada modelo em todas as imagens e compara com o gabarito (prompt 1).
4. **Decisão:** a equipe escolhe o modelo. Atualize `config.json`, rode `testar_modelo.py`, gere notas novas coerentes com as imagens validadas e refaça os exemplos de `docs/`.
5. **Se nenhum modelo servir:** levar à equipe a decisão sobre ajuste fino, que hoje está fora de escopo, ou sobre fotografar as ferramentas em condições parecidas com as do treino.

Pronto quando: existe um modelo com contagem exata em pelo menos duas imagens que sirvam a um cenário conforme e a um divergente, o `config.json` está atualizado e os testes passam.

> **Prompt 1 — avaliação de modelos**
>
> Leia `AGENTS.md` e as seções 3, 4 e 7.2 do `CONTINUIDADE.md`. Crie `src/avaliar_modelos.py`, que recebe uma lista de `model_id` por `--modelos` e, opcionalmente, um mapeamento de classes por modelo (JSON) para as nossas classes `Pliers`, `Screwdriver`, `Wrench` e `Hammer`. Para cada modelo e cada imagem de `imagens/teste/`, o script chama o Roboflow reaproveitando `detector.criar_cliente` e `detector.converter_resposta` (troque só o `model_id` numa cópia do config). Ele conta as detecções com confiança de pelo menos 0,5 e compara com `imagens/teste/gabarito.json`. Gere `saidas/avaliacao/<modelo>.json` e imprima uma tabela por modelo com: imagens com contagem exata, total de itens detectados corretamente, faltas e excessos. Salve uma resposta bruta de cada modelo em `fixtures/` antes de assumir o formato. Não altere `config.json` nem o contrato do relatório. Inclua testes sem rede para a lógica de comparação com o gabarito.

> **Prompt 1b — adotar o modelo escolhido**
>
> Troque o modelo para `<model_id>` no `config.json` e ajuste `de_para`, `nomes_exibicao` e `classes_ignoradas` às classes dele. Rode `src/testar_modelo.py` e me mostre um resumo por imagem. Depois, com as imagens que eu indicar, reescreva `notas/nota_conforme.json` e `notas/nota_divergente.json` (a divergente declara uma unidade a mais do que a imagem mostra), regenere as evidências de `docs/` e atualize as seções de limitações do `README.md` e do `CONTINUIDADE.md`.

7.3 Passo 2 — Modo câmera

Objetivo: criar `src/camera.py`. Ele mostra o vídeo ao vivo; a tecla espaço congela o quadro e roda exatamente o mesmo fluxo da conferência (`conferir_quadro`); Q sai. **Os quadros nunca são somados.**

Requisitos:

- Uso: `.venv\Scripts\python.exe src\camera.py --nota notas/nota_divergente.json [--camera 0]`.
- No Windows, abra a câmera com `cv2.VideoCapture(indice, cv2.CAP_DSHOW)`. Se ela não abrir, mostre uma mensagem clara e saia com o código 2.
- A nota é validada uma vez, na abertura. Uma nota inválida mostra a mensagem e o programa não chega a abrir a câmera.
- Ao apertar espaço:
  – roda `conferir_quadro`;
  – grava `relatorio.json`, `imagem_anotada.jpg` e `quadro_original.jpg` numa pasta nova `saidas/camera_<data_hora>/`;
  – mostra a evidência numa segunda janela;
  – imprime o status no terminal.
- Enquanto a chamada ao Roboflow acontece, o vídeo mostra "Conferindo…". Uma chamada síncrona serve.
- Falha de rede vira status `erro` (já tratado em `conferir_quadro`) e o programa continua aberto.
- Para testar sem webcam, separe a lógica em funções puras (por exemplo, `processar_captura(quadro, nota, config, pasta)`) e teste com quadros sintéticos e clientes falsos.

Sem ferramentas físicas: dá para testar apontando a câmera para fotos de ferramentas na tela de um celular.

Pronto quando: abre, captura, gera as três saídas, não soma quadros, os testes sem rede passam e o README tem uma seção "Modo câmera".

Prompt 2 — modo câmera

Leia `AGENTS.md` e as seções 3 e 7.3 do `CONTINUIDADE.md`. Implemente `src/camera.py` conforme os requisitos da seção 7.3, reaproveitando `conferir_quadro`, `montar_evidencia` e `salvar_jpg`, sem duplicar lógica. Mantenha a lógica testável fora do laço de vídeo e escreva `tests/test_camera.py` sem rede e sem webcam (use quadros numpy sintéticos e um cliente falso). Atualize o README com uma seção "Modo câmera". Rode os testes e me mostre o resultado.

7.4 Passo 3 — Integração com o sistema do colega

Objetivo: ligar esta conferência ao sistema que o colega está construindo, com um orquestrador comum aos dois.

Primeiro, respondam juntos:

1. Em que linguagem e ambiente o outro sistema roda? Na mesma máquina?
2. Como a imagem chega: caminho de arquivo, bytes, URL ou câmera?
3. Como a nota chega: arquivo JSON, objeto em memória ou outro formato, como XML de NF-e?
4. O que o orquestrador precisa de volta: só o status, o relatório inteiro ou também a imagem de evidência?
5. A chamada é síncrona (espera a resposta) ou por fila ou pasta?

Escolha a forma de integração pela resposta:

Situação	Forma recomendada	Já existe?
Orquestrador em Python, mesmo processo	Importar `conferir()` ou `conferir_quadro()`	Sim
Outra linguagem, mesma máquina	CLI com `--json` e o código de saída	Sim
Troca por arquivos	Ler `relatorio.json` e `imagem_anotada.jpg` da pasta `--saida`	Sim
Outra máquina ou serviço web	API HTTP (por exemplo, FastAPI com `POST /conferir`)	Não; precisa do aval da equipe, pois estava fora de escopo

Regras da integração:

- **Não quebre o contrato do relatório** (seção 3.6). Campos novos são permitidos; renomear ou remover campos não é.
- Se a nota real tiver outro formato, crie um **conversor** para o formato da seção 3.3 em vez de mudar o `nota.py`.
- Códigos de produto reais entram no `de_para` do `config.json`.
- Escreva um teste de integração com cliente falso.

Pronto quando: o orquestrador chama a conferência, recebe o relatório e trata os três status, incluindo `erro`, e existe um teste de integração sem rede.

Prompt 3a — integração local

Leia `AGENTS.md` e as seções 3 e 7.4 do `CONTINUIDADE.md`. O outro sistema é `<descreva: linguagem, como envia imagem e nota, o que espera de volta>`. Crie o ponto de integração usando `conferir_quadro` ou a CLI com `--json`, sem mudar o contrato do relatório. Se a nota vier em outro formato, crie `src/conversor_nota.py`, que a converte para o formato da seção 3.3, com testes. Escreva um teste de integração com cliente falso cobrindo os três status (`conforme`, `divergente`, `erro`).

> **Prompt 3b — API HTTP (só com o aval da equipe)**
>
> Leia `AGENTS.md` e as seções 3 e 7.4 do `CONTINUIDADE.md`. Crie `src/api.py` com FastAPI e um endpoint `POST /conferir` que recebe uma imagem (multipart) e a nota (JSON) e devolve o relatório da seção 3.6, com a evidência em base64 num campo opcional `imagem_anotada_base64`. Reaproveite `validar_nota` e `conferir_quadro`. Nota inválida responde HTTP 422 com a mensagem; falha técnica responde 200 com `status: "erro"`, como no restante do sistema. Fixe as versões no `requirements.txt` depois de instalar. Escreva testes com o `TestClient` do FastAPI e um cliente falso do Roboflow. Documente no README como subir a API.

---

8. Checklist de entrega final

- ☐ `AGENTS.md` criado (prompt 0)
- ☐ Gabarito das imagens escrito por uma pessoa
- ☐ Modelo avaliado e escolhido; `config.json` atualizado
- ☐ Notas e evidências de `docs/` refeitas com o modelo novo
- ☐ Modo câmera funcionando (se houver webcam)
- ☐ Integração com o sistema do colega, com teste de integração
- ☐ `pytest` passando
- ☐ README atualizado (uso, câmera, integração e limitações)
- ☐ Saídas de reserva guardadas fora de `saidas/`, para o caso de falta de internet na apresentação
- ☐ `.env` fora do repositório e chave ausente de qualquer arquivo versionado

9. Crédito

Modelo "Tools Classification", de **proelec3** (Roboflow Universe), licença CC BY 4.0: https://universe.roboflow.com/proelec3/tools-classification-byznk. Se o modelo for trocado, dê crédito ao novo autor no README conforme a licença dele.